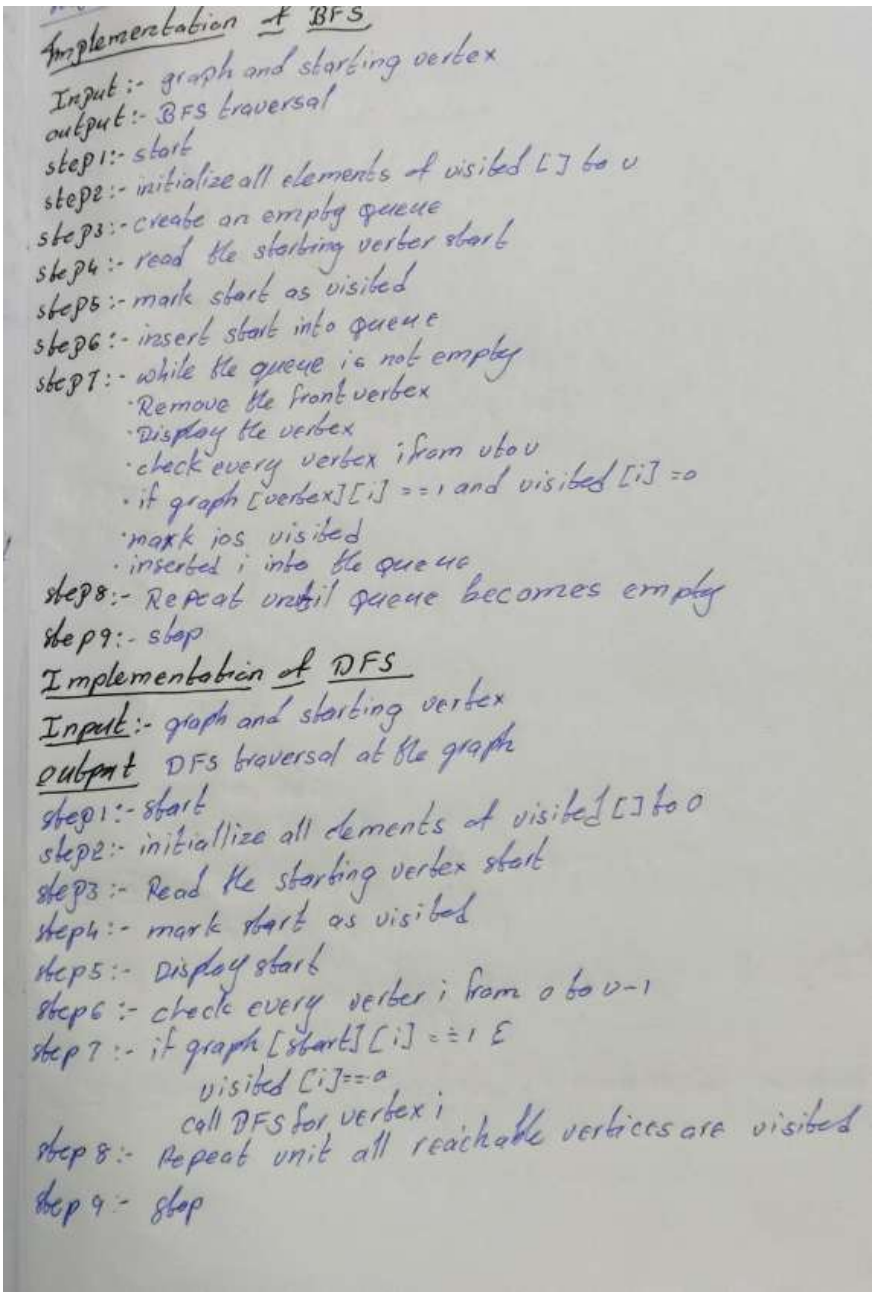
PRACTICAL: 08

AIM:- Implementation of Graph and Searching (DFS and BFS).

Algorithms:-



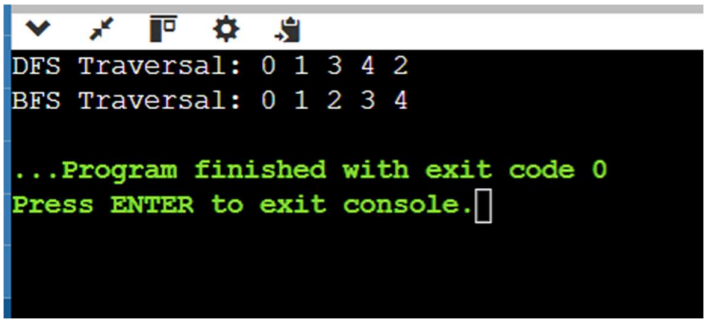
Program/code:-

```
#include <iostream>
#include<queue>
using namespace std;
int main()
{
 // Static adjacency matrix
 int graph[5][5] =
 {
 {0, 1, 1, 0, 0},
 {1, 0, 0, 1, 1},
 {1, 0, 0, 0, 0},
 {0, 1, 0, 0, 0},
 {0, 1, 0, 0, 0}
 };
 int visited[5] = {0};
 // ---------- DFS ----------
cout << "DFS Traversal: ";
 int stack[5];
int top = -1;
 stack[++top] = 0;
 visited[0] = 1;
 while (top >= 0)
 {
 int vertex = stack[top--];
```

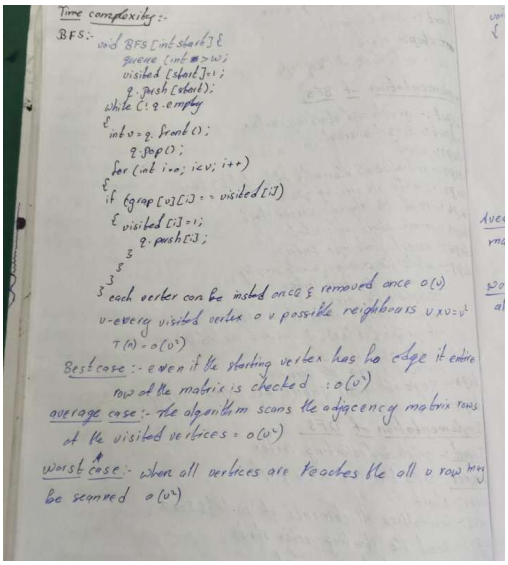
```
 cout << vertex << " ";
 // Add adjacent vertices
for (int i = 4; i >= 0; i--)
 {
 if (graph[vertex][i] == 1 && visited[i] == 0)
 {
visited[i] = 1;
 stack[++top] = i;
 }
 }
 }
 // Reset visited array
 for (int i = 0; i < 5; i++)
visited[i] = 0;
 // ---------- BFS ----------
cout << "\nBFS Traversal: ";
 queue<int> q;
 q.push(0);
 visited[0] = 1;
 while (!q.empty())
 {
 int vertex = q.front();
q.pop();
 cout << vertex << " ";
 for (int i = 0; i < 5; i++)
 {
 if (graph[vertex][i] == 1 && visited[i] == 0)
```

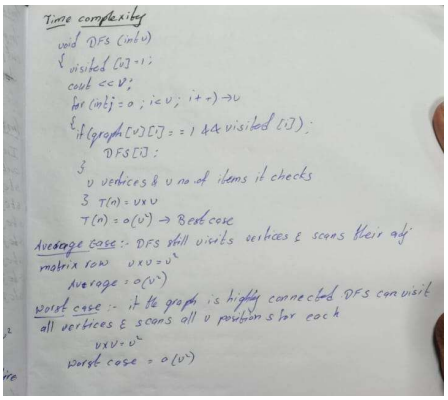
```
 {
 visited[i] = 1;
q.push(i);
 }
 }
 }
 return 0;
}
```

Output:-



Time complexity:-





Conclusion:-

Graphs organize connected nodes using an adjacency list, mapping vertices directly to their neighboring nodes.

BFS uses a queue to explore neighbors level-by-level, making it ideal for finding the shortest path in unweighted networks.

DFS uses a stack (or recursion) to explore deep down a single branch first, making it the top choice for cycle detection and maze solving.